Fundamental Analysis Platform

Complete Technical Documentation

Table of Contents

System Overview

Platform Purpose

A SaaS web application that provides institutional-grade fundamental analysis and equity research capabilities. Users upload financial statements or search for companies, receive AI-generated analysis reports, and conduct interactive Q&A with the results. Premium users gain access to stock screening engines with custom filtering.

Key Features

- **Core Analysis**: Upload financial documents → AI-powered fundamental analysis report
- **Interactive Q&A**: Chat interface for follow-up questions on reports
- **Stock Screening**: Premium feature for filtering equities by custom criteria
- **Multi-format Support**: PDF, Excel, CSV financial statements
- **Real-time Data**: Integration with financial APIs for current stock metrics
- **Report Storage**: Historical analysis with revision tracking

Target Users

- **Basic Tier**: Retail investors, students, financial enthusiasts
- **Premium Tier**: Fund managers, portfolio analysts, institutional researchers
- **Enterprise**: Financial advisory firms, brokerage platforms

---

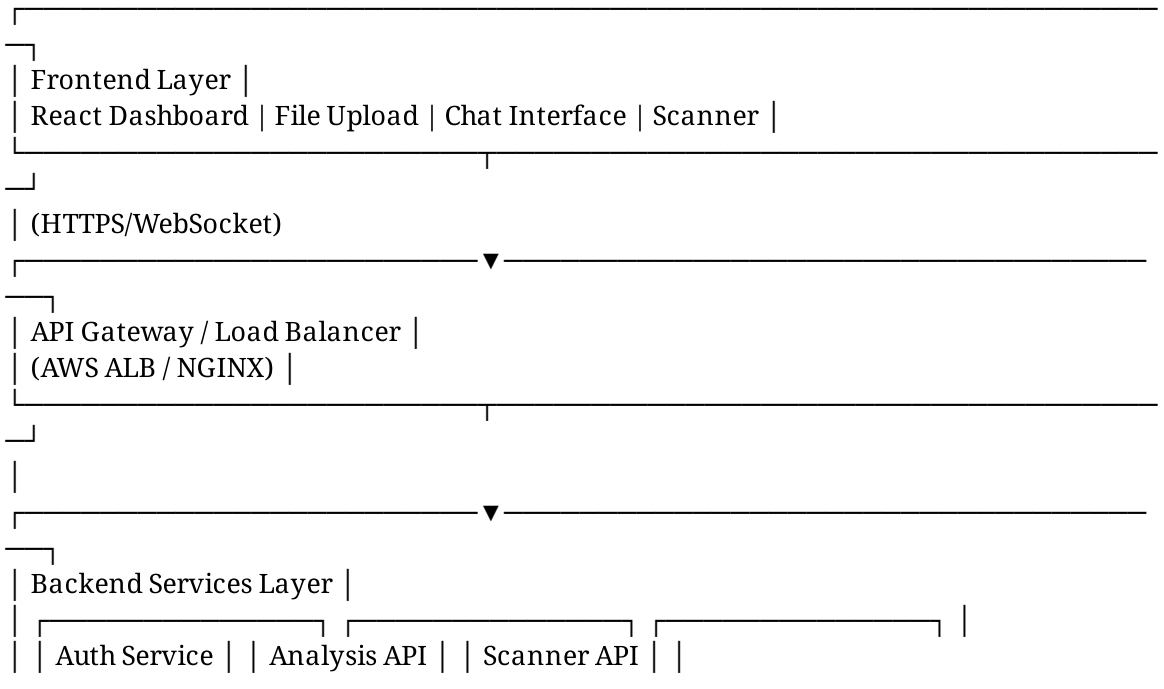
Architecture & Tech Stack

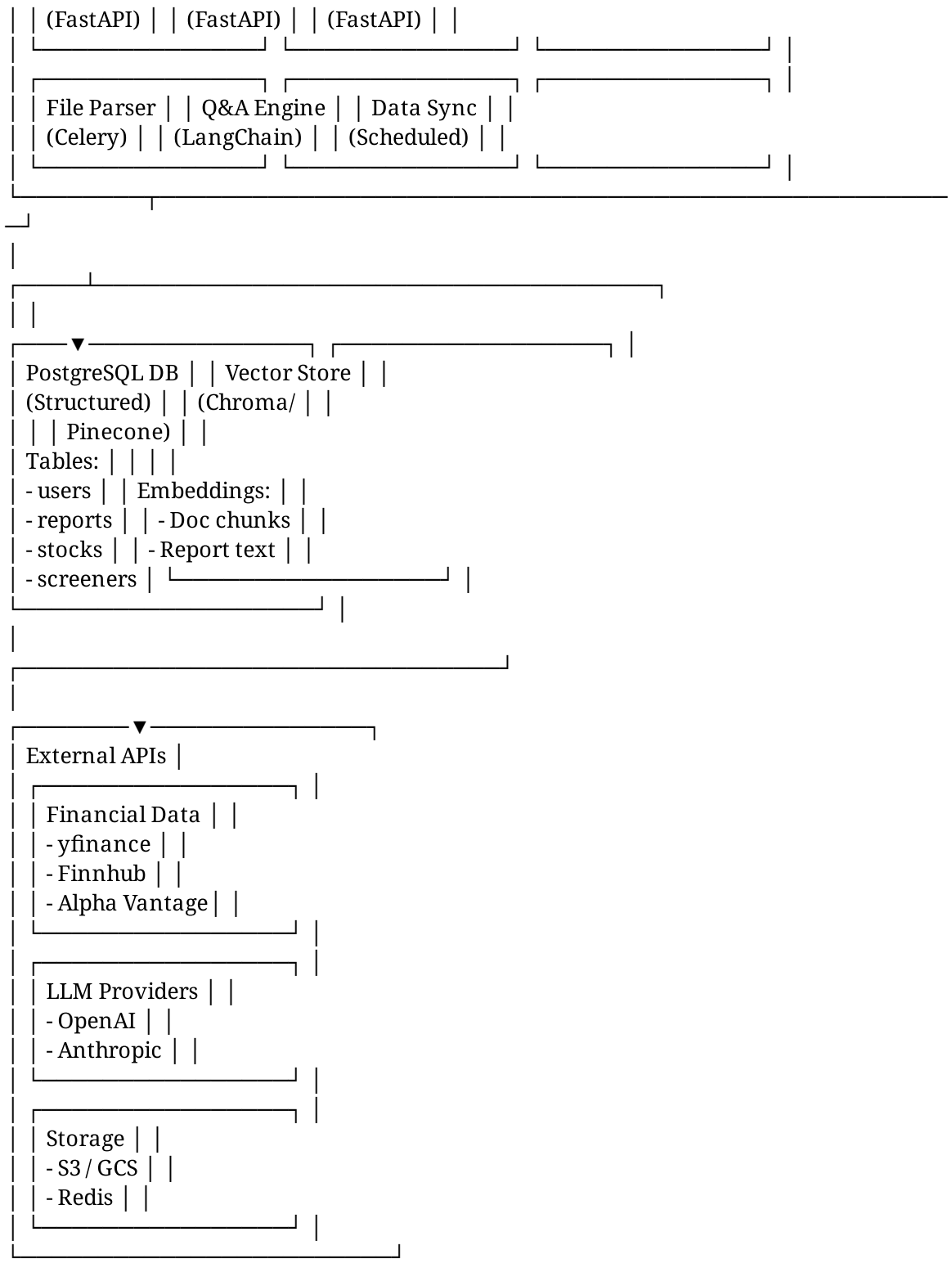
Technology Choices

Component	Technology	Rationale
Frontend	React 18 + TypeScript	Component reusability, strong typing, ecosystem maturity
	TailwindCSS	Rapid styling, design system consistency, low CSS payload
	Zustand/Redux	State management for report cache and user sessions
Backend	FastAPI (Python 3.11+)	Async I/O, native OpenAPI docs, rapid prototyping
	Pydantic	Request validation, schema generation
Database	PostgreSQL 15+	ACID compliance, JSON support for flexible schemas
	Alembic	Database migrations, version control
Vector Store	Chroma / Pinecone	Semantic search for RAG, embedded docs retrieval
File Processing	PyPDF2 / pdfplumber	PDF extraction, table detection
	pandas / openpyxl	Excel/CSV parsing, data transformation
Financial Data	yfinance	Free stock price & historical data
	Alpha Vantage / EODHD	Enhanced metrics (for premium tier)
	SEC EDGAR API	US financial statements (10-K, 10-Q)
	Finnhub / FMP API	Real-time quotes, screening data
LLM & NLP	LangChain	Orchestration, prompt management, chains

Component	Technology	Rationale
	OpenAI GPT-4 / Claude 3	Primary LLM for analysis generation
	Ollama (optional)	On-premise Llama 2/3 for data privacy
Authentication	Auth0 / Firebase	OAuth 2.0, multi-factor authentication
Task Queue	Celery + Redis	Async report generation, PDF parsing
Caching	Redis	Session management, rate limiting
Deployment	AWS (EC2/ECS) or Docker/Kubernetes	Containerized microservices
Storage	AWS S3 / Google Cloud Storage	Document storage, report exports
Monitoring	Datadog / Sentry	Error tracking, performance metrics

System Architecture Diagram




(FastAPI)
(FastAPI)
(FastAPI)
File Parser
(Celery)
Q&A Engine
(LangChain)
Data Sync
(Scheduled)
PostgreSQL DB
(Structured)
Tables:
- users
- reports
- stocks
- screeners
Vector Store
(Chroma/
Pinecone)
Embeddings:
- Doc chunks
- Report text
External APIs
Financial Data
- yfinance
- Finnhub
- Alpha Vantage
LLM Providers
- OpenAI
- Anthropic
Storage
- S3 / GCS
- Redis

Database Schema

Users Table

```
CREATE TABLE users (
id SERIAL PRIMARY KEY,
email VARCHAR(255) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
full_name VARCHAR(255),
tier VARCHAR(20) DEFAULT 'free', -- 'free', 'premium', 'enterprise'
subscription_status VARCHAR(20) DEFAULT 'inactive',
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
metadata JSONB -- Custom user preferences
);
```

Reports Table

```
CREATE TABLE reports (
id SERIAL PRIMARY KEY,
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
company_name VARCHAR(255) NOT NULL,
ticker VARCHAR(10),
report_type VARCHAR(50), -- 'fundamental', 'technical', 'custom'
report_content TEXT NOT NULL, -- Full LLM-generated report
summary TEXT, -- Short summary for display
analysis_data JSONB, -- Structured metrics (ratios, scores)
source_document_id INTEGER REFERENCES source_documents(id),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
is_public BOOLEAN DEFAULT FALSE,
version INTEGER DEFAULT 1,
INDEX idx_user_created (user_id, created_at)
);
```

Source Documents Table

```
CREATE TABLE source_documents (
id SERIAL PRIMARY KEY,
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
filename VARCHAR(500),
file_type VARCHAR(20), -- 'pdf', 'excel', 'csv'
file_path VARCHAR(1000), -- S3 path
file_size_bytes INTEGER,
extracted_text TEXT, -- Raw extracted content
metadata JSONB, -- Document info: period, company, filing type
upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
processing_status VARCHAR(50), -- 'pending', 'processed', 'failed'
INDEX idx_user_upload (user_id, upload_date)
);
```

Stock Data Table

```
CREATE TABLE stocks (
id SERIAL PRIMARY KEY,
ticker VARCHAR(10) UNIQUE NOT NULL,
company_name VARCHAR(255) NOT NULL,
exchange VARCHAR(20), -- 'NSE', 'BSE', 'NYSE', 'NASDAQ'
sector VARCHAR(100),
industry VARCHAR(100),
current_price DECIMAL(15, 2),
market_cap_usd BIGINT,
pe_ratio DECIMAL(10, 2),
dividend_yield DECIMAL(5, 2),
revenue_growth_yoy DECIMAL(5, 2),
profit_margin DECIMAL(5, 2),
debt_to_equity DECIMAL(5, 2),
current_ratio DECIMAL(5, 2),
roe DECIMAL(5, 2),
eps DECIMAL(10, 2),
last_updated TIMESTAMP,
data_source VARCHAR(50), -- 'yfinance', 'finnhub', 'custom'
UNIQUE(ticker, exchange),
INDEX idx_sector (sector),
INDEX idx_metrics (pe_ratio, dividend_yield, revenue_growth_yoy)
);
```

Screeners Table

```
CREATE TABLE screeners (
id SERIAL PRIMARY KEY,
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
name VARCHAR(255) NOT NULL,
description TEXT,
filters JSONB NOT NULL, -- {pe_ratio_max: 20, revenue_growth_min: 10, ...}
results JSONB, -- Cached results
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
last_run TIMESTAMP,
is_saved BOOLEAN DEFAULT TRUE,
UNIQUE(user_id, name)
);
```

Chat Sessions Table

```
CREATE TABLE chat_sessions (
id SERIAL PRIMARY KEY,
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
report_id INTEGER NOT NULL REFERENCES reports(id) ON DELETE CASCADE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
conversation_history JSONB, -- Array of {role, content, timestamp}
context_chunks TEXT, -- Relevant doc chunks for RAG
```

INDEX idx_user_report (user_id, report_id)
);

API Specification

Base URL

https://api.fundamentalanalysis.io/v1

Authentication

All endpoints require Bearer token in header:
Authorization: Bearer <jwt_token>

Core Endpoints

1. Authentication

POST /auth/register
Request:
{
"email": "user@example.com",
"password": "secure_password_123",
"full_name": "John Analyst"
}

Response (201):
{
"user_id": 1,
"email": "user@example.com",
"access_token": "eyJhbGc...",
"refresh_token": "eyJhbGc...",
"tier": "free"
}

POST /auth/login
Request:
{
"email": "user@example.com",
"password": "secure_password_123"
}

Response (200):
{
"access_token": "eyJhbGc...",
"refresh_token": "eyJhbGc...",
"user": { "id": 1, "email": "...", "tier": "premium" }
}

2. Document Upload & Analysis

POST /reports/upload
Request (multipart/form-data):
{
"file": <binary_file>,
"company_name": "Apple Inc.", -- optional if ticker provided
"ticker": "AAPL" -- optional
}

Response (202):
{
"task_id": "abc-123-def",
"report_id": 42,
"status": "processing",
"message": "Analysis queued. Check back in 30-60 seconds.",
"polling_url": "/reports/42/status"
}

GET /reports/{report_id}/status
Response (200):
{
"report_id": 42,
"status": "completed", -- 'processing', 'completed', 'failed'
"progress": 100,
"result": {
"company_name": "Apple Inc.",
"ticker": "AAPL",
"report_url": "/reports/42"
}
}

GET /reports/{report_id}
Response (200):
{
"id": 42,
"company_name": "Apple Inc.",
"ticker": "AAPL",
"summary": "Apple shows strong profitability with...",
"full_report": "## Financial Health Overview\n\nApple demonstrates...",
"analysis_data": {
"profitability_score": 8.5,
"liquidity_score": 7.8,
"solvency_score": 8.2,
"key_ratios": {
"current_ratio": 1.08,
"quick_ratio": 1.05,
"debt_to_equity": 1.98,
"roe": 89.6,
"gross_margin": 46.2
},
"red_flags": ["High debt levels", "Slowing revenue growth"],

"strengths": ["Dominant market share", "Strong cash generation"]
},
"created_at": "2025-01-02T14:30:00Z",
"updated_at": "2025-01-02T14:30:00Z"
}

3. Company Search (Free Data)

GET /companies/search?q={query}
Request:
GET /companies/search?q=apple

Response (200):
[
{
"ticker": "AAPL",
"company_name": "Apple Inc.",
"exchange": "NASDAQ",
"sector": "Technology",
"market_cap_usd": 2800000000000,
"data_available": true
}
]

GET /companies/{ticker}
Response (200):
{
"ticker": "AAPL",
"company_name": "Apple Inc.",
"sector": "Technology",
"exchange": "NASDAQ",
"current_price": 225.50,
"market_cap_usd": 2800000000000,
"pe_ratio": 28.5,
"dividend_yield": 0.46,
"revenue_growth_yoy": 2.1,
"profit_margin": 25.5,
"debt_to_equity": 1.98,
"current_ratio": 1.08,
"roe": 89.6,
"eps": 6.05,
"last_updated": "2025-01-02T16:00:00Z"
}

4. Interactive Q&A

POST /reports/{report_id}/chat
Request:
{
"question": "Why did revenue decline in Q3?",
"session_id": "sess_xyz" -- optional, creates new if not provided
}

Response (200):
{
"session_id": "sess_xyz",
"question": "Why did revenue decline in Q3?",
"answer": "Based on the financial statements, revenue declined due to: 1) Supply chain disruptions in semiconductor production... 2) Currency headwinds in international markets... Reference: 10-Q filing page 5, Management Discussion section.",
"confidence": 0.92,
"sources": [
{
"type": "financial_statement",
"page": 5,
"chunk": "Supply chain challenges..."
}
],
"timestamp": "2025-01-02T14:35:00Z"
}

GET /reports/{report_id}/chat/history
Response (200):
[
{
"role": "user",
"content": "What is the debt level?",
"timestamp": "2025-01-02T14:30:00Z"
},
{
"role": "assistant",
"content": "Total debt is $123B...",
"timestamp": "2025-01-02T14:30:30Z"
}
]

5. Stock Screening (Premium)

POST /screeners/create
Request:
{
"name": "Growth Tech Stocks",
"filters": {
"sector": "Technology",
"market_cap_min": 10000000000, -- $10B
"pe_ratio_max": 30,
"revenue_growth_min": 15, -- 15% YoY
"profit_margin_min": 10
}
}

Response (201):
{
"screener_id": 5,
"name": "Growth Tech Stocks",

"status": "created"
}

GET /screeners/{screener_id}/run
Response (200):
{
"screener_id": 5,
"name": "Growth Tech Stocks",
"filter_count": 5,
"results": [
{
"ticker": "MSFT",
"company_name": "Microsoft Corporation",
"sector": "Technology",
"current_price": 415.25,
"market_cap_usd": 3100000000000,
"pe_ratio": 28.5,
"revenue_growth_yoy": 16.2,
"profit_margin": 35.8,
"match_score": 0.95
},
{
"ticker": "NVDA",
"company_name": "NVIDIA Corporation",
"sector": "Technology",
"current_price": 880.25,
"market_cap_usd": 2200000000000,
"pe_ratio": 65.2,
"revenue_growth_yoy": 126.0,
"profit_margin": 52.1,
"match_score": 0.88
}
],
"total_matches": 2,
"last_run": "2025-01-02T16:00:00Z"
}

GET /screeners
Response (200):
[
{
"id": 5,
"name": "Growth Tech Stocks",
"created_at": "2025-01-01T10:00:00Z",
"last_run": "2025-01-02T16:00:00Z",
"match_count": 2
}
]

6. Report Export

GET /reports/{report_id}/export?format=pdf
Response (200): Binary PDF file
Content-Disposition: attachment; filename="AAPL_Analysis_2025-01-02.pdf"

Frontend Components

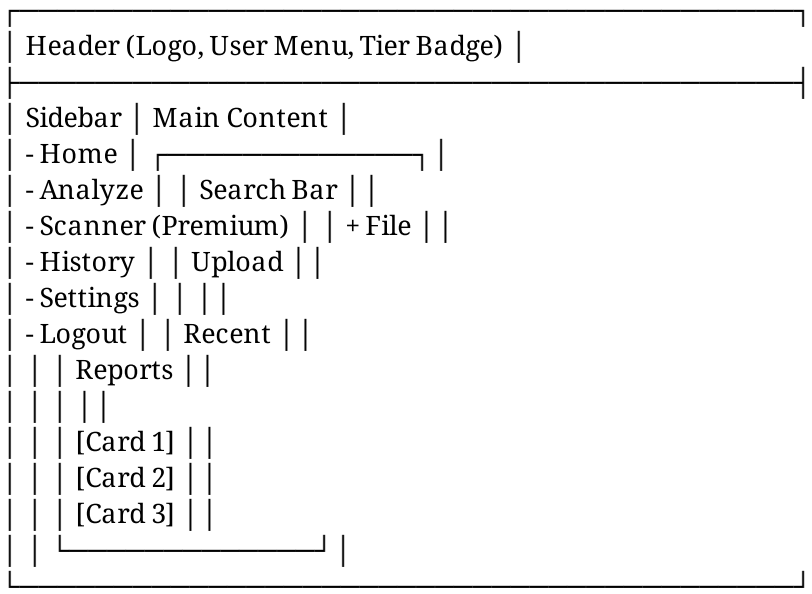
Page Structure

1. Landing Page

- Hero section with platform features
- Sign-up / Log-in buttons
- Use case examples
- Pricing tiers table

2. Dashboard (Authenticated)

Layout:

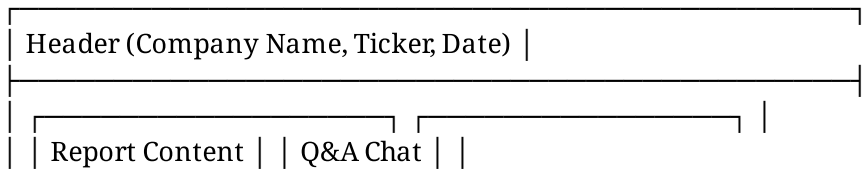


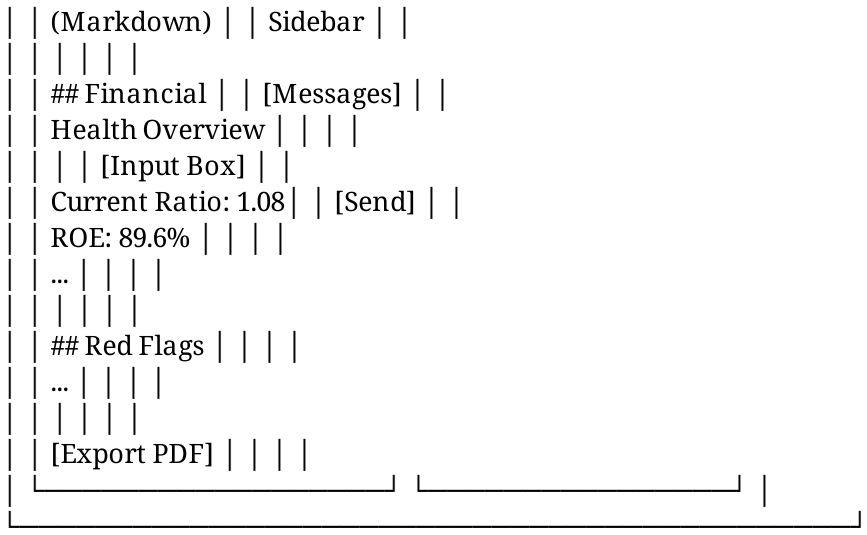
Key Components:

- **SearchBar**: Company name / ticker autocomplete
- **FileUpload**: Drag-drop or click-to-upload
- **ReportCard**: Shows company, date, summary, action buttons
- **RecentReports**: Grid or list view of past analyses

3. Analysis Report Page

Layout:



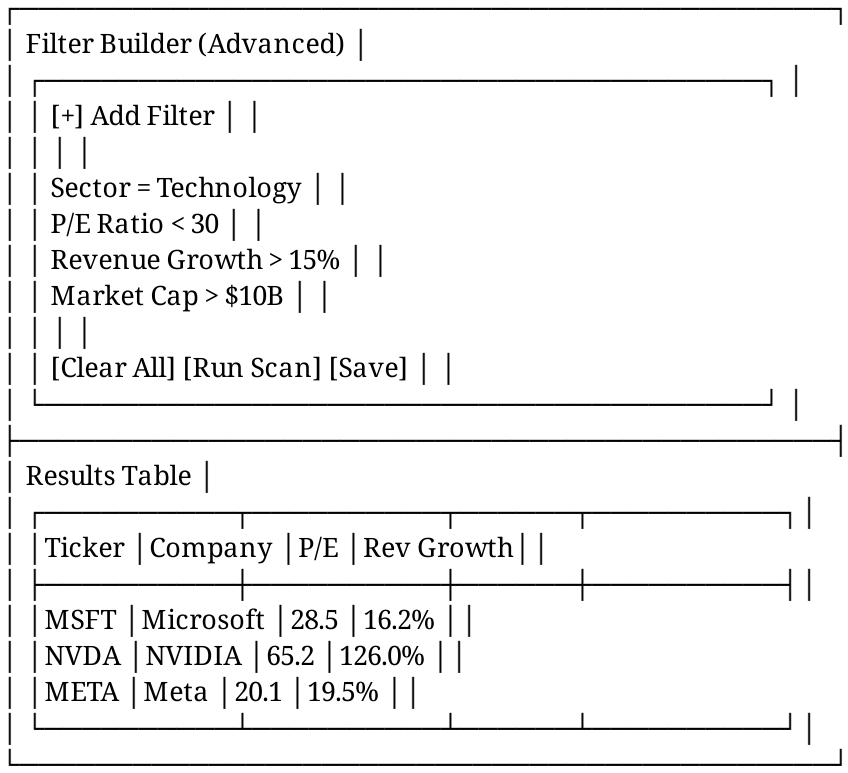


Key Components:

- **ReportContent**: Rendered markdown with rich formatting
- **MetricsPanel**: Key ratios displayed in cards or tabs
- **ChatSidebar**: Message history with auto-scroll
- **InputBox**: Textarea with send button, typing indicator
- **ExportButton**: Download as PDF

4. Stock Scanner (Premium)

Layout:



Key Components:

- **FilterBuilder**: Drag-drop filters, operators
- **FilterChip**: Individual filter with delete button
- **ResultsTable**: Sortable, filterable columns
- **SaveTemplate**: Store and reuse screeners

5. User Settings

- **Profile**: Email, name, avatar upload
- **Subscription**: Current tier, upgrade button, billing history
- **API Keys**: For programmatic access (future)
- **Integrations**: Connected services

React Component Tree

```
App
├── Auth
│   ├── LoginPage
│   └── RegisterPage
├── Layout
│   ├── Header
│   ├── Sidebar
│   └── MainContent
│   ├── Dashboard
│   │   ├── SearchBar
│   │   ├── FileUpload
│   │   └── ReportCardList
│   ├── AnalysisReport
│   │   ├── ReportViewer
│   │   │   ├── MetricsPanel
│   │   │   └── MarkdownContent
│   │   └── ChatSidebar
│   │   ├── MessageList
│   │   └── InputBox
│   ├── StockScanner (Premium)
│   │   ├── FilterBuilder
│   │   ├── ResultsTable
│   │   └── SaveTemplate
│   └── Settings
│   ├── ProfileForm
│   ├── SubscriptionInfo
│   └── APIKeys
└── Providers
├── AuthContext
├── ThemeProvider
└── ToastNotifications
```

AI/LLM Integration

System Prompt for Fundamental Analysis

You are an expert fundamental analysis financial analyst with 15+ years of experience in equity research and financial statement analysis.

Your role is to analyze company financial statements and provide actionable, professional-grade fundamental analysis reports.

When you receive financial data about a company:

1. FINANCIAL HEALTH OVERVIEW
    - Assess profitability (gross margin, net margin, operating margin trends)
    - Evaluate liquidity (current ratio, quick ratio, cash position)
    - Analyze solvency (debt levels, interest coverage, debt-to-equity)
    - Comment on cash flow generation and quality of earnings
2. RATIO ANALYSIS
    - Calculate and interpret key ratios:
        - Profitability: ROE, ROA, ROIC, profit margins (gross, operating, net)
        - Liquidity: current ratio, quick ratio, working capital ratio
        - Leverage: debt-to-equity, net debt-to-EBITDA, interest coverage
        - Efficiency: asset turnover, receivables turnover, inventory turnover
        - Valuation: P/E ratio, price-to-book, EV/EBITDA
    - Compare to industry averages (provide if available)
    - Identify trends (3-5 year analysis if available)
3. PERFORMANCE DRIVERS
    - Explain revenue growth/decline drivers
    - Analyze margin expansion/compression causes
    - Identify one-time items or exceptional gains/losses
    - Assess competitive positioning and market share
4. RED FLAGS & CONCERNS
    - Declining profitability or margins
    - Deteriorating balance sheet quality
    - Negative cash flow despite reported earnings
    - High debt levels relative to peers
    - Unusual accounting practices or related-party transactions
    - Litigation or regulatory risks
5. STRENGTHS & OPPORTUNITIES
    - Competitive advantages and moats
    - Growth opportunities in new markets or products
    - Pricing power and brand value
    - Strong cash generation and shareholder returns
6. INVESTMENT ASSESSMENT
    - Risk-reward profile assessment
    - Valuation conclusion (overvalued/undervalued/fairly valued)
    - Suitable investor profiles
    - Key metrics to monitor going forward

FORMAT REQUIREMENTS:

- Use clear section headings with markdown formatting

- Include specific numbers and percentages in analysis
- Provide context: "Revenue grew 15% YoY, above the industry average of 8%"
- Include a 2-3 sentence executive summary at the start
- Use bullet points for clarity
- Keep language professional but accessible

Always include disclaimers:

- "This analysis is for educational purposes only"
- "Not financial advice; conduct further research before investing"
- "Analysis based on available data; may not reflect latest developments"

Prompt for Q&A Follow-ups

You are a financial analyst assistant. A user is asking follow-up questions about a company's fundamental analysis report and financial statements.

Instructions:

1. Answer the user's question concisely (2-3 paragraphs max)
2. Reference specific numbers from the documents
3. Cite which document/section you're referencing
4. If the answer requires information not in the provided documents, state: "This information is not available in the provided documents. I recommend checking [specific source]."
5. Maintain a professional, educational tone
6. Do not provide investment recommendations

Context (relevant chunks from financial documents and report):
{context}

User Question: {question}

Answer:

LangChain Chains

Document Analysis Chain:

```
from langchain.chains import StuffDocumentsChain
from langchain.prompts import PromptTemplate
from langchain.llms import ChatOpenAI

llm = ChatOpenAI(model_name="gpt-4", temperature=0.2)

analysis_prompt = PromptTemplate(
input_variables=["page_content", "company_name"],
template="""
Analyze this financial statement for {company_name}:
{page_content}
```

Provide structured fundamental analysis...

```
    """
```

)

chain = StuffDocumentsChain(
llm_chain=LLMChain(llm=llm, prompt=analysis_prompt),
document_variable_name="page_content",
document_prompt=PromptTemplate(...)
)

Q&A with RAG Chain:
from langchain.chains import RetrievalQA
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

embeddings = OpenAIEmbeddings()
vectorstore = Chroma(persist_directory="./chroma_db")

qa_chain = RetrievalQA.from_chain_type(
llm=llm,
chain_type="stuff",
retriever=vectorstore.as_retriever(search_kwargs={"k": 5}),
return_source_documents=True
)

result = qa_chain({"query": "Why did revenue decline?"})

Data Pipeline

1. File Ingestion Flow

User Upload
↓
FileValidator (check file type, size, format)
↓
S3 Upload (persist for audit trail)
↓
Queue Job (Celery task)
↓
FileParser (PDF → text, Excel → structured data)
↓
TextCleaning (remove headers/footers, normalize)
↓
MetadataExtraction (filing date, company, period)
↓
TextChunking (split into 500-token chunks with overlap)
↓
VectorEmbeddings (generate embeddings for each chunk)
↓
Store in Chroma (vector DB for retrieval)
↓

Store raw text in PostgreSQL
↓
LLMAnalysisJob (trigger report generation)
↓
Completion Webhook → Frontend notification

2. Report Generation Flow

Financial Data + Extracted Text
↓
LLM Analysis Prompt
↓
GPT-4 Processes
↓
Structured Output (parse into JSON)
↓
Format Report (markdown with headings)
↓
Store in PostgreSQL (reports table)
↓
Index Summary (for search)
↓
Notify User

3. Stock Data Sync

Scheduled every 6 hours:

Pseudocode

```
for ticker in popular_tickers:
data = yfinance.download(ticker)
metrics = calculate_ratios(data)
db.stocks.update_or_create(
ticker=ticker,
fields={
'current_price': data['close'][-1],
'pe_ratio': metrics['pe'],
'revenue_growth_yoy': metrics['rev_growth'],
...
}
)
db.commit()
```

Premium Features

1. Stock Screening Engine

Architecture:

- Real-time filter application against `stocks` table
- Support for 20+ financial metrics
- Compound filters with AND/OR logic

Supported Filters:
Market Cap Range: [$5B, $2T]
P/E Ratio Range: [5, 100]
Dividend Yield: [0%, 10%]
Revenue Growth (YoY): [-50%, +200%]
Profit Margin: [0%, 50%]
ROE: [0%, 100%]
Debt-to-Equity: [0, 5]
Current Ratio: [0.5, 3]
Sector: [Technology, Finance, Healthcare, ...]
Price-to-Book: [0.5, 5]
EV/EBITDA: [5, 30]

API Integration:

- Cache results for 1 hour
- Return top 100 matches
- Rank by relevance score

2. Scanner Templates & Alerts

Save Templates:
POST /screeners/save
{
"name": "Value Dividend Stocks",
"description": "Low P/E, high dividend yield",
"filters": {...}
}

Email Alerts (Future):

- Notify when new stocks match saved criteria
- Daily/weekly summary emails
- Set price targets for alerts

3. Advanced Analytics (Future)

- Backtesting screeners against historical data
- Correlation analysis between stocks
- Portfolio analytics and benchmarking

Deployment & Scaling

Docker Containerization

Dockerfile (FastAPI backend):
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

Docker Compose (local development):
version: '3.8'

services:
backend:
build: ./backend
ports:
- "8000:8000"
environment:
DATABASE_URL: postgresql://user:pass@postgres:5432/fundanalysis
REDIS_URL: redis://redis:6379
OPENAI_API_KEY: ${OPENAI_API_KEY}
depends_on:
- postgres
- redis
- chroma

frontend:
build: ./frontend
ports:
- "3000:3000"

postgres:
image: postgres:15
environment:
POSTGRES_DB: fundanalysis
POSTGRES_USER: user
POSTGRES_PASSWORD: pass
volumes:
- postgres_data:/var/lib/postgresql/data

redis:
image: redis:7-alpine

chroma:
image: ghcr.io/chroma-core/chroma:latest
ports:

- "8001:8000"
volumes:
- chroma_data:/chroma/data

volumes:
postgres_data:
chroma_data:

AWS Deployment

Infrastructure:

- **Compute**: ECS Fargate (serverless containers) or EC2 with auto-scaling
- **Database**: RDS PostgreSQL (multi-AZ for HA)
- **Caching**: ElastiCache Redis
- **Storage**: S3 for documents, CloudFront for CDN
- **API**: API Gateway + ALB
- **Monitoring**: CloudWatch, X-Ray

Scaling Strategy:

1. **Horizontal Scaling**: Increase Fargate task replicas (CPU/memory triggers)
2. **Database**: RDS read replicas for analytics queries
3. **Async Jobs**: SQS + Celery workers (scale workers independently)
4. **Caching**: Redis for session/results cache

Kubernetes Deployment (Long-term)

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: backend
spec:
replicas: 3
selector:
matchLabels:
app: backend
template:
metadata:
labels:
app: backend
spec:
containers:
- name: backend
image: registry.example.com/backend:latest
ports:
- containerPort: 8000
env:
- name: DATABASE_URL
valueFrom:
secretKeyRef:
name: db-secret
key: url
resources:
requests:
memory: "512Mi"
cpu: "500m"
limits:
memory: "1Gi"
cpu: "1000m"
livenessProbe:
```

httpGet:
path: /health
port: 8000
initialDelaySeconds: 10
periodSeconds: 10

apiVersion: v1
kind: Service
metadata:
name: backend-service
spec:
selector:
app: backend
ports:

- protocol: TCP
  port: 80
  targetPort: 8000
  type: LoadBalancer

Security & Compliance

Authentication & Authorization

Implement:

- OAuth 2.0 via Auth0 or Firebase
- JWT tokens with 1-hour expiry, refresh token rotation
- Multi-factor authentication (optional, free tier; required premium)
- Role-based access control (RBAC): user, premium_user, admin

Endpoints requiring authentication:

- All `/reports`, `/screeners`, `/user` endpoints
- Public: `/auth`, `/companies/search` (with rate limiting)

Data Security

At Rest:

- Encrypt sensitive columns (passwords, API keys) with AES-256
- S3 bucket encryption enabled
- Database encryption at rest

In Transit:

- TLS 1.3 for all API communication
- HTTPS enforced (redirect HTTP to HTTPS)
- Secure WebSocket (WSS) for chat

File Handling:

- Scan uploaded files for malware (VirusTotal API)
- Restrict file types (PDF, XLS, XLSX only)
- Max file size: 50 MB
- Validate file headers (magic bytes)

GDPR & Data Privacy

Compliance:

- Data deletion on account termination (GDPR "right to be forgotten")
- Data export functionality (GDPR data portability)
- Privacy policy clearly stating data use (LLM training: opt-out available)
- Data residency: Option to store in specific regions
- Regular security audits and penetration testing

Regulatory Disclaimers

Display prominently:
⚠ Disclaimer
This platform provides educational financial analysis for informational purposes only. It does not constitute investment advice, recommendations, or offers to buy/sell securities.

- Past performance is not indicative of future results
- Always conduct independent research before investing
- Consult a licensed financial advisor for personalized guidance
- The platform is not liable for investment losses

This analysis is generated by AI and may contain errors. Users assume all risk related to investment decisions made based on this platform.

API Rate Limiting

Per user limits

FREE_TIER: 5 requests/minute, 100 requests/day
PREMIUM_TIER: 50 requests/minute, 10,000 requests/day
ENTERPRISE: Unlimited

Implement via Redis

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.get("/reports/{report_id}")
@limiter.limit("5/minute")
```

```
def get_report(report_id: int):
...
```

Development Roadmap

Phase 1: MVP (Weeks 1-8)

- [ ] User authentication (Auth0)
- [ ] File upload + PDF/Excel parsing
- [ ] Basic LLM integration for analysis
- [ ] Report display with markdown rendering
- [ ] Simple chat Q&A
- [ ] PostgreSQL + Chroma setup

Phase 2: Core Features (Weeks 9-16)

- [ ] Premium tier with stock screening
- [ ] Real-time stock data integration (yfinance)
- [ ] Advanced RAG with multi-document context
- [ ] Report history and search
- [ ] Celery async task queue
- [ ] Redis caching

Phase 3: Scale & Polish (Weeks 17-24)

- [ ] Kubernetes deployment
- [ ] Multi-region support
- [ ] Advanced analytics (backtesting, correlation)
- [ ] Email alerts for screeners
- [ ] API key access for premium users
- [ ] White-label option for enterprises

Phase 4: Enterprise Features (Q3 2025+)

- [ ] Sentiment analysis integration
- [ ] Custom model fine-tuning
- [ ] Integration with portfolio management systems
- [ ] Advanced compliance tooling (for advisors)

Monitoring & Observability

Key Metrics

System Health:

- API response time (p95, p99)
- Error rate (4xx, 5xx responses)
- Database query time (slow query log)
- Background job processing time

Business Metrics:

- Reports generated per day
- Average time from upload to completion
- User retention (DAU, MAU)
- Premium tier conversion rate

LLM Performance:

- Token usage and costs
- Report generation success rate
- Average confidence scores for Q&A

Logging

```
import logging
from pythonjsonlogger import jsonlogger

logger = logging.getLogger()
handler = logging.StreamHandler()
formatter = jsonlogger.JsonFormatter()
handler.setFormatter(formatter)
logger.addHandler(handler)
```

Structured logging

```
logger.info("Report generated", extra={
"user_id": 123,
"report_id": 42,
"duration_ms": 45000,
"tokens_used": 2500
})
```

Error Tracking (Sentry)

```
import sentry_sdk

sentry_sdk.init(
dsn="https://key@sentry.io/123456",
environment="production",
traces_sample_rate=0.1
)
```

Errors automatically captured

Testing Strategy

Unit Tests

tests/test_analysis.py

def test_extract_financial_ratios():
data = {
"current_assets": 1000,
"current_liabilities": 500,
}
assert extract_ratios(data)["current_ratio"] == 2.0

def test_llm_prompt_formatting():
prompt = format_analysis_prompt(financial_data)
assert "Financial Health Overview" in prompt

Integration Tests

tests/test_api.py

def test_report_generation_e2e(test_client, sample_pdf):
response = test_client.post("/reports/upload", files={"file": sample_pdf})
assert response.status_code == 202

```
task_id = response.json()["task_id"]
report = wait_for_completion(task_id)
assert report["status"] == "completed"
```

Load Testing

Using locust

locust -f tests/locustfile.py --host=https://api.example.com

Conclusion

This technical documentation provides a comprehensive blueprint for building the Fundamental Analysis Platform. Key highlights:

1. **Scalable Architecture**: Microservices, containerized, cloud-ready
2. **AI-Driven**: LangChain + LLM for intelligent analysis and Q&A
3. **Data-Rich**: Multiple financial data sources, vector embeddings for RAG
4. **Compliant**: GDPR, regulatory disclaimers, data privacy
5. **Production-Ready**: Monitoring, testing, security best practices

For questions or clarifications on any section, refer to the API Specification (Section 4) for exact payload structures, or the Architecture diagram (Section 2) for system interactions.

Appendix: Key Dependencies

Backend (Python):
fastapi0.104.1
pydantic2.4.2
sqlalchemy2.0.23
alembic1.12.1
psycopg2-binary2.9.9
redis5.0.1
celery5.3.4
langchain0.1.0
openai1.3.9
pdfplumber0.10.3
pandas2.1.3
yfinance0.2.32

Frontend (React):
react18.2.0
react-router-dom6.20.0
zustand4.4.1
tailwindcss3.3.6
axios1.6.2
markdown-it-react0.1.2
react-markdown9.0.1
react-hot-toast2.4.1
recharts==2.10.3

DevOps:
docker
docker-compose
kubernetes
terraform